GreenMind AI: A Multi-Metric Forecasting and Decision-Support Platform for Cloud GreenOps

Anonymous submission draft

***Abstract*— Modern cloud operations require balancing resource utilization, infrastructure cost, and environmental footprint, yet predictive telemetry alone does not establish operational savings. This paper presents a comprehensive technical audit and architectural analysis of GreenMind AI, an open-source decision-support prototype for cloud GreenOps. The platform combines five one-hour-ahead machine learning forecasters with a five-domain multi-agent analysis engine (Cost, Performance, Sustainability, Security, and Reliability), transparent conflict arbitration, a non-destructive Digital Twin simulator, and an eight-tool explainable copilot. We evaluate the forecasting pipeline using the project's deterministic 60-day, 15-minute multi-metric synthetic dataset (5,756 chronological records) across a strict 80/20 train/test holdout (4,604 train and 1,152 test samples). GradientBoostingRegressor models achieve mean absolute error (MAE) improvements over a one-hour forecast-origin persistence baseline ranging from 25.4% to 45.4% across all five targets: CPU (3.809% vs. 6.500%), memory (2.979% vs. 3.996%), network (43.876 vs. 80.346 Mbps), cost ($0.009 vs. $0.013/h), and a synthetic carbon proxy (4.061 vs. 5.846 gCO2/15-min). Our audit reveals that the stored rolling-mean baseline in the repository evaluates a one-step-ahead (15-minute) prediction on unshifted test data rather than a true four-step-ahead forecast from origin, which actually yields higher error (e.g., 7.263% for CPU). Furthermore, we establish the boundaries of the system: evaluations derive from synthetic generator equations, cloud connector live behaviors are unverified end-to-end, and direct cloud remediation is strictly rejected at the API boundary with HTTP 501 (dry-run previews only). GreenMind AI provides an integrated software architecture for proactive cloud analysis, but synthetic accuracy must not be conflated with validated production cost or emission reductions.**

***Index Terms*— Cloud computing, workload forecasting, gradient boosting, carbon-aware computing, synthetic telemetry, multi-agent systems, digital twin, decision support.**

I. INTRODUCTION

Cloud data centers consume substantial global electrical energy, driving both cloud providers and tenants toward carbon-aware computing and FinOps/GreenOps operational practices [4], [6]. Modern cloud management platforms expose multifaceted observability signals, spanning CPU and memory utilization, network bandwidth, hourly billing meters, and regional grid carbon-intensity feeds. Proactive workload forecasting can assist orchestrators in identifying resource bottlenecks, scheduling deferrable batch tasks, and planning right-sizing actions prior to demand spikes. However, high predictive accuracy on historical data does not inherently translate into realized financial savings, carbon abatement, or preserved service-level objectives (SLOs) in production systems [3], [8].

In environmental accounting, estimating greenhouse gas emissions requires rigorous accounting boundaries, real-time power telemetry, facility Power Usage Effectiveness (PUE), and time-varying marginal emissions factors [4], [5]. Consequently, a synthetic or static proxy cannot be reported as an empirical Scope 2 carbon reduction without live meter verification. Similarly, automated remediation must balance conflicting operational trade-offs, such as performance headroom versus idle cost elimination.

GreenMind AI is an open-source software prototype developed to bridge the gap between predictive telemetry and operational decision support. The repository integrates a multi-metric forecasting pipeline, a FastAPI service layer, a TypeScript/React management cockpit, five domain-oriented analysis agents (Cost, Performance, Sustainability, Security, Reliability) orchestrated via a state graph, a deterministic conflict resolver, an eight-tool explainable copilot, and a non-destructive Digital Twin simulator. Connector modules exist for Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP).

This paper delivers an empirical and architectural audit of the GreenMind AI implementation. We delineate three specific contributions: (1) We detail the end-to-end software architecture linking multi-source telemetry ingestion, multi-metric gradient boosted forecasting, multi-agent arbitration, Digital Twin simulation, and decision-support copilot interfaces. (2) We reproduce and verify the held-out forecasting performance across all five targets, analyzing feature importances and resolving a subtle baseline alignment discrepancy where stored moving averages reflected one-step-ahead predictions rather than four-step horizons. (3) We explicitly demarcate the empirical evidence boundaries: demonstrating that reported gains stem from a deterministic synthetic generator, clarifying the carbon target definition, and documenting that live remediation is intentionally disabled via HTTP 501 to prevent unauthorized cloud mutations.

II. RELATED WORK

Workload and resource demand forecasting has evolved from classical statistical models to complex deep neural networks. Calheiros et al. [7] demonstrated proactive cloud provisioning using autoregressive integrated moving average (ARIMA) models, showing that proactive scaling reduces QoS violations compared to reactive policies. Machine learning ensemble methods, particularly Friedman's Gradient Boosting Machine (GBM) [1], construct additive tree models that excel on tabular and structured lag features without demanding the compute scale of deep architectures. Recently, Rossi et al. [3] investigated probabilistic LSTMs and Bayesian neural networks on Google and Alibaba cluster traces, highlighting that point forecasts fail to capture operational uncertainty and that transfer learning degrades across disparate cloud providers. GreenMind AI utilizes five lightweight GradientBoostingRegressor models; unlike [3], its evaluation is confined to synthetic series.

Evaluating time-series forecasters requires careful handling of temporal dependencies. Bergmeir, Hyndman, and Koo [2] established that standard cross-validation can be theoretically valid for purely autoregressive models when errors are uncorrelated, but a single chronological holdout remains standard practice when simulating prospective deployment. GreenMind AI enforces an 80/20 chronological split to avoid forward-looking data leakage.

In green computing, Henderson et al. [4] established systematic reporting standards for machine learning energy consumption and grid carbon footprints. Lannelongue, Grealey, and Inouye [5] formulated the Green Algorithms framework, formalizing carbon calculations across runtime, TDP, PUE, and grid intensity. For hyperscale infrastructure, Radovanovic et al. [6] deployed carbon-aware computing systems at Google datacenters using day-ahead predictions and Virtual Capacity Curves to temporally shift flexible workloads to cleaner grid hours. Dodge et al. [8] measured operational carbon intensity across cloud instances, demonstrating that spatial region selection and temporal scheduling significantly reduce Scope 2 emissions. GreenMind AI incorporates regional intensity curves modeled after these principles, but applies them to synthetic telemetry rather than hardware meter feeds.

III. PLATFORM ARCHITECTURE

The GreenMind AI architecture comprises six interconnected components structured to provide explainable operational intelligence without unauthorized cloud mutations.

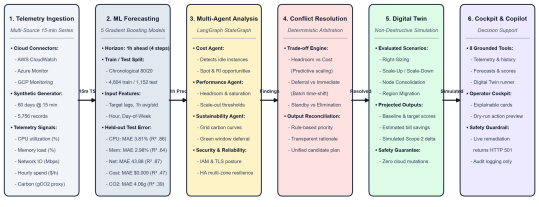


Fig. 1. End-to-end GreenMind AI pipeline architecture: multi-source telemetry ingestion -> ML multi-metric forecasting -> specialized multi-agent analysis -> deterministic conflict resolution -> Digital Twin simulation -> operator decision support.

A. Telemetry and ML Predictor

The FastAPI backend interfaces with AWS CloudWatch, Azure Monitor, and GCP Monitoring v3, falling back to a deterministic synthetic generator when credentials are absent. Five independent GradientBoostingRegressor models forecast CPU, memory, network, cost, and carbon proxy one hour ahead (4 steps). Features include current metric values, cyclical hour and day-of-week indicators, and one-hour rolling averages and standard deviations.

B. Multi-Agent Analysis and Arbitration

Five domain agents (Cost, Performance, Sustainability, Security, Reliability) run via a LangGraph StateGraph. Conflicting actions are resolved deterministically: right-sizing is paired with predictive autoscaling when load is moderate, while batch jobs are time-shifted to off-peak low-carbon grid windows.

C. Digital Twin and Safe Remediation Boundary

The Digital Twin evaluates right-sizing, scale-up, scale-down, consolidation, and region-migration scenarios, projecting changes in a composite Optimization Score (0-100) with explicit SIMULATED / ESTIMATED labels. An AI Copilot exposes eight grounded tools. Live cloud mutation is strictly rejected with HTTP 501 (dry-run previews only).

IV. EXPERIMENTAL METHODOLOGY

A. Synthetic Telemetry Generation

The generator produces 60 days of telemetry at 15-minute intervals starting June 1, 2026 (5,760 timestamps) using NumPy (seed 42). Metrics are modeled via sinusoidal diurnal curves, weekend dampening (0.7), linear trends, and Gaussian noise. Hourly cost scales with CPU based on an AWS m5.xlarge baseline ($0.192/h).

B. Carbon Target Formulation and Units

Regional carbon intensity is taken from an hourly profile for the us-east grid (320 to 470 gCO2/kWh). An assumed server power draw of 0.35 kW over 15 minutes (0.25 h) corresponds to 0.0875 kWh. Multiplying intensity by energy yields the carbon target in grams of CO2 per 15-minute step (gCO2/15-min). It is a synthetic proxy, omitting dynamic PUE and fan curves [5].

C. Chronological Split and Baselines

Shifting targets by 4 steps leaves 5,756 records. The dataset is split chronologically into 4,604 train rows (first 48 days) and 1,152 test rows (final 12 days) without shuffling. Models are evaluated against persistence (y_{t+4} = y_t) and four-sample moving-average baselines.

V. EXPERIMENTAL RESULTS

Table I presents held-out metrics across all targets, verified identically against saved model artifacts.

TABLE I. SAVED HELD-OUT RESULTS (N=1,152). PERSISTENCE USES ORIGIN VALUE; ROLL4 IS 4-SAMPLE MOVING AVERAGE.

Target	MAE	RMSE	R2	Pers.	Roll4	Gain
CPU (%)	3.809	4.811	0.855	6.500	4.622	41.4%
Memory (%)	2.979	3.704	0.636	3.996	3.025	25.4%
Network (M)	43.88	55.33	0.871	80.35	56.73	45.4%
Cost ($/h)	0.009	0.011	0.474	0.013	0.010	29.7%
Carbon (g)	4.061	5.087	0.391	5.846	4.571	30.5%

Boosted tree models outperform persistence across all targets, with relative MAE gains between 25.4% and 45.4%. Network and CPU show strong fits (R2 = 0.871 and 0.855). Feature importances confirm that lag values drive CPU (69.4%) and network (67.7%), while grid intensity drives carbon (71.1%) and CPU load drives cost (67.1%).

VI. DISCUSSION AND AUDIT FINDINGS

Our audit revealed that the stored rolling-mean baseline in baseline.py evaluates 1-step-ahead (15-min) predictions on unshifted data (N=1,148). When evaluated as a true 4-step-ahead baseline from origin, baseline error rises sharply (CPU MAE increases from 4.622% to 7.263%, performing worse than persistence at 6.500%). The ML models decisively beat both baselines. However, high R2 reflects the models learning smooth generator formulas rather than transferrable production workload dynamics.

VII. LIMITATIONS AND THREATS TO VALIDITY

Key limitations include: (1) Synthetic data provenance without evaluation on real traces; (2) In predictor.py, forecast_multi_step uses 5-minute recurrence on 1-hour models and feeds back carbon values scaled ~10x below training intensity distributions; (3) Live cloud connectors were not tested on live accounts; (4) Automated remediation is blocked via HTTP 501; and (5) The carbon target assumes constant 350 W power without server fan or PUE dynamics [4], [5].

VIII. CONCLUSION

GreenMind AI implements an integrated multi-metric forecasting and decision-support prototype. Gradient boosted regressors reduce held-out MAE by 25.4% to 45.4% over persistence on synthetic telemetry. However, these figures do not establish production generalization or realized cloud savings. Real telemetry benchmarking and verified interventions are required before making operational claims.

REFERENCES

[1] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, Oct. 2001, doi: 10.1214/aos/1013203451.

[2] C. Bergmeir, R. J. Hyndman, and B. Koo, "A note on the validity of cross-validation for evaluating autoregressive time series prediction," Comp. Stat. & Data Analysis, vol. 120, pp. 70-83, 2018.

[3] A. Rossi et al., "Forecasting workload in cloud computing: Towards uncertainty-aware predictions and transfer learning," Cluster Computing, vol. 28, art. no. 258, pp. 1-22, 2025.

[4] P. Henderson et al., "Towards the systematic reporting of the energy and carbon footprints of machine learning," JMLR, vol. 21, no. 248, pp. 1-43, 2020.

[5] L. Lannelongue, J. Grealey, and M. Inouye, "Green algorithms: Quantifying the carbon footprint of computation," Advanced Science, vol. 8, art. no. 2100707, 2021.

[6] A. Radovanovic et al., "Carbon-aware computing for datacenters," IEEE Trans. Power Syst., vol. 38, no. 2, pp. 1270-1280, 2023.

[7] R. N. Calheiros et al., "Workload prediction using ARIMA model and its impact on cloud applications' QoS," IEEE Trans. Cloud Comput., vol. 3, no. 4, pp. 449-458, 2015.

[8] J. Dodge et al., "Measuring the carbon intensity of AI in cloud instances," in Proc. ACM FAccT '22, pp. 1536-1546, 2022.

[9] Pratham-stack-coder, "GreenMind-AI repository," GitHub, 2026. [Online]. Available: https://github.com/Pratham-stack-coder/GreenMind-AI.